Sistema de enrutamiento para tiendas de autoservicio

Alumnos:

- Lara Beltran Jorge Alberto
- Pacheco Pérez Diego

Docente: Zuriel Dathan Mora Félix

Materia: Tópicos selectos de Inteligencia Artificial

1. Introducción

En este proyecto se aborda el problema de optimización de rutas de distribución, el cual es fundamental en logística y transporte. El objetivo principal es minimizar el costo total considerando distancia y consumo de combustible.

Para resolver este problema, se implementó el algoritmo de Recocido Simulado (Simulated Annealing), una técnica metaheurística que permite encontrar soluciones cercanas al óptimo en problemas complejos.

2. Descripción del Problema

El problema corresponde al Vehicle Routing Problem (VRP), donde múltiples vehículos deben distribuir productos desde centros hacia tiendas.

Características:

- Múltiples rutas (una por vehículo)
- Cada ruta inicia y termina en un centro
- Las tiendas se distribuyen entre rutas
- Se minimiza el costo total

3. Representación del Problema

Las rutas se representan como listas de nodos:

```
rutas = [
  [1, 5, 8, 1],  # Ruta vehículo 1
  [2, 6, 9, 2],  # Ruta vehículo 2
]
```

Donde:

- El primer y último nodo son el centro
- Los valores intermedios son tiendas

Inicialización de rutas

```python
def inicializar_rutas(self, num_vehiculos, centros_idx):
        num_nodos = self.matriz_distancias.shape[0]
        tiendas_idx = list(range(max(centros_idx) + 1, num_nodos + 1))
        np.random.shuffle(tiendas_idx)

        rutas = np.array_split(tiendas_idx, num_vehiculos)
        for i in range(num_vehiculos):
            rutas[i] = [centros_idx[i]] + list(rutas[i]) + [centros_idx[i]]
        return rutas
```

4. Función de Costo

La función objetivo combina varios factores:

- Distancia
- Combustible
- Penalización adicional

De forma general:

$$Costo = \alpha * distancia + \beta * combustible + \varepsilon * penalización$$

Donde:

- α = peso de la distancia
- β = peso del combustible
- ε = factor de penalización

```
def calcular_costo(self, rutas, alpha=1.0, beta=1.0, epsilon=0.1):
        costo_total = 0

        for ruta in rutas:
            origenes = np.array(ruta[:-1]) - 1
            destinos = np.array(ruta[1:]) - 1

            for o, d in zip(origenes, destinos):
                distancia = self.distancias[o, d]
                combustible = self.combustible[o, d]

                costo_tramo = alpha * distancia + beta * combustible

                costo_tramo += epsilon * (distancia + combustible)

                costo_total += costo_tramo

        return costo_total
```

5. Generación de Vecindario

Se generan soluciones vecinas intercambiando tiendas entre rutas:

```
def generar_vecino(self, rutas):
        nuevas_rutas = [r[:] for r in rutas]

        indices = np.arange(len(nuevas_rutas))
        np.random.shuffle(indices)
        r1, r2 = indices[:2]

        ruta1, ruta2 = nuevas_rutas[r1], nuevas_rutas[r2]

        if len(ruta1) > 2 and len(ruta2) > 2:
            i = np.random.randint(1, len(ruta1) - 1)
            j = np.random.randint(1, len(ruta2) - 1)

            temp = ruta1[i]
            ruta1[i] = ruta2[j]
            ruta2[j] = temp

            nuevas_rutas[r1], nuevas_rutas[r2] = ruta1, ruta2

        return nuevas_rutas
```

6. Algoritmo de Recocido Simulado

```
def recocidoSimulado(self):
        """Ejecuta el proceso de recocido."""
        actual = self.rutas
        mejor = self.rutas
        costo_actual = self.calcular_costo(actual)
        costo_mejor = costo_actual
        sin_mejora = 0

        while self.T > self.T_min:
            for _ in range(self.iteraciones):
                vecino = self.generar_vecino(actual)
                costo_vecino = self.calcular_costo(vecino)
                delta = costo_vecino - costo_actual

                if delta < 0 or np.random.rand() < math.exp(-delta /
self.T):
                    actual = vecino
                    costo_actual = costo_vecino
                    if costo_actual < costo_mejor:
                        mejor = actual
                        costo_mejor = costo_actual
                        sin_mejora = 0
                    else:
                        sin_mejora += 1

            if sin_mejora > 500:
                self.T *= 0.9
                sin_mejora = 0
            else:
                self.T *= self.alpha

            print(f"Temperatura: {self.T:.4f} | Mejor Costo:
{costo_mejor:.2f}")

        return mejor, costo_mejor
```

7. Visualización de Rutas

Las rutas optimizadas se grafican utilizando coordenadas geográficas:

```
def graficar_rutas(self):
        """Grafica las rutas optimizadas."""
        coords = self.datos.ubicaciones

        plt.figure()

        # Dibujar rutas
        for i, ruta in enumerate(self.resultado_rutas):
            x = []
            y = []

            for idx in ruta:
                fila = coords.iloc[idx - 1]
                x.append(fila["Longitud_WGS84"])
                y.append(fila["Latitud_WGS84"])

            plt.plot(x, y, marker='o', label=f'Ruta {i+1}')
…
```

8. Resultados

El algoritmo produce:

- Rutas optimizadas
- Costo total mínimo

Ejemplo:

```
Costo total óptimo: 12345.67
Ruta 01 -> Centro -> Tienda A -> Tienda B -> Centro
Ruta 02 -> Centro -> Tienda C -> Tienda D -> Centro
```

9. Análisis

- Se logra reducir el costo global
- Las rutas se distribuyen eficientemente
- El algoritmo evita óptimos locales

Importante:

Se optimiza el conjunto de rutas, no una ruta individual

10. Conclusiones

- El Recocido Simulado es adecuado para problemas complejos
- Permite explorar múltiples soluciones
- Es flexible y escalable